INFORMATION RETRIEVAL

Practicals 6 to 10

Code - Explanation - Theory - Viva Questions with Answers

Computer Laboratory-II (BE Semester I) - Department of Artificial Intelligence and Data Science, DYPIEMR, Pune

No.	Practical	Key concept
6	Text Pre-processing: Stop Word Removal and Stemming	Tokenization, stop words, stemming
7	Retrieval of Documents using Inverted Files (Inverted Index)	Inverted index, Boolean retrieval
8	Bayesian Network for Medical Data (Heart Disease Diagnosis)	Bayesian network, probabilistic inference
9	Agglomerative Hierarchical Clustering	Agglomerative clustering, dendrogram
10	PageRank Algorithm	Link analysis, PageRank

How to use this guide

- Each practical has: **Aim, Theory, Algorithm, Complete Code, Code Explanation, Output, Conclusion** and **Viva Questions with Answers**.
- All programs were run and tested. Practicals 6, 7 and 10 show real output. Practicals 8 and 9 need your data files (heart.csv and CC_GENERAL.csv), so only the output format is shown.
- Install the libraries once: `pip install nltk pandas numpy pgmpy networkx scikit-learn scipy matplotlib`
- Write the Aim, Algorithm, Code, Output and Conclusion in your journal. Learn the Theory and the viva answers in your own words.
- A final section has general Information Retrieval questions that examiners often ask.

Practical 6: Text Pre-processing: Stop Word Removal and Stemming

Aim: Write a program for pre-processing of a text document such as stop word removal and stemming.

Software / Libraries: Python 3, Jupyter Notebook, NLTK (pip install nltk)

1. Theory

Text pre-processing converts raw text into a clean, standard form that an information retrieval (IR) system or machine learning model can use. Raw text contains punctuation, capital letters, very common words and many word forms (run, running, runs). Pre-processing reduces this noise, shrinks the index size and improves retrieval quality.

Main pre-processing steps

- **Tokenization** - splitting a document into individual words (tokens). Example: 'Hello world!' becomes ['Hello', 'world', '!'].
- **Case folding (lower-casing)** - converting all letters to lower case so that 'Computer' and 'computer' are treated as the same term.
- **Punctuation / noise removal** - removing symbols, numbers and special characters that carry no meaning for retrieval.
- **Stop word removal** - deleting very frequent words that carry little meaning.
- **Stemming / lemmatization** - reducing each word to its base or root form.

Stop words

A **stop word** is a very common word (the, a, an, in, is, of, and) that a search engine ignores while indexing and while searching. Removing them saves storage and processing time, because such words appear in almost every document and do not help to distinguish one document from another. NLTK stores stop word lists for about 16 languages in *nltk_data/corpora/stopwords*. The list can be edited by adding or removing words.

Stemming

Stemming is the process of producing the morphological variants of a root word by chopping off suffixes using fixed rules. For example, 'retrieval', 'retrieved' and 'retrieves' all reduce to the stem 'retriev'. The stem need not be a valid dictionary word. The input to a stemmer is a list of tokens, so tokenization must be done first.

Stemmer	Key idea	Remarks
Porter (1980)	Removes suffixes in several rule-based steps (e.g. -ing, -ed, -ational)	Most popular, fast, English only
Snowball (Porter2)	Improved Porter algorithm, supports many languages	More accurate and faster than Porter
Lancaster	Aggressive iterative rule set stored externally	Fastest but over-stems the most
Lovins (1968)	Removes the longest matching suffix, then repairs the stem	Oldest stemmer, rarely used now

Errors in stemming

- **Over-stemming (false positive)** - two words with different meanings are reduced to the same stem. Example: 'university' and 'universe' both become 'univers'.
- **Under-stemming (false negative)** - two words with the same meaning are NOT reduced to the same stem. Example: 'data' and 'datum' stay different.

Lemmatization is an alternative to stemming. It uses a dictionary and the word's part of speech to return a valid base word (the lemma), e.g. 'better' becomes 'good' and 'running' becomes 'run'. It is more accurate but slower than stemming.

Applications

Search engines and indexing, text classification, sentiment analysis, document clustering, text summarization and domain vocabulary analysis.

2. Algorithm

1. Read or define the input text (string or file).
2. Tokenize the text into words.
3. Convert tokens to lower case and remove punctuation.
4. Remove stop words using the NLTK English stop word list.
5. Apply a stemmer (Porter / Snowball / Lancaster) to every remaining word.
6. Print or write the pre-processed words to an output file.

3. Program (Python)

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer, SnowballStemmer, LancasterStemmer

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')

text = ("Programmers program with programming languages. "
        "The quick brown foxes are jumping over the lazy dogs!")

# Step 1: Tokenization
tokens = word_tokenize(text)
print("Tokens          :", tokens)

# Step 2: Lowercase + remove punctuation
words = [w.lower() for w in tokens if w.isalnum()]
print("Cleaned words   :", words)

# Step 3: Stop word removal
stop_words = set(stopwords.words('english'))
filtered = [w for w in words if w not in stop_words]
print("After stop words:", filtered)

# Step 4: Stemming with three different stemmers
porter = PorterStemmer()
snowball = SnowballStemmer('english')
lancaster = LancasterStemmer()

print("\n{:<14}{:<12}{:<12}{:<12}".format("Word", "Porter", "Snowball", "Lancaster"))
for w in filtered:
    print("{:<14}{:<12}{:<12}{:<12}".format(
        w, porter.stem(w), snowball.stem(w), lancaster.stem(w)))

# Step 5: Doing the same on a text file (text.txt -> filteredtext.txt)
with open("text.txt", "w") as f:
    f.write("This is a sample file showing off the stop words filtration.")

with open("text.txt") as f:
    file_words = word_tokenize(f.read())

result = [porter.stem(w) for w in file_words
          if w.lower() not in stop_words and w.isalnum()]

with open("filteredtext.txt", "w") as f:
    f.write(" ".join(result))
print("\nFile output     :", " ".join(result))
```

4. Code Explanation

Code / block	What it does
import nltk ... from nltk.stem import ...	Imports tokenizer, stop word corpus and three stemmers. nltk.download() fetches the 'punkt' tokenizer models and the stop word lists the first time the program runs.
word_tokenize(text)	Splits the sentence into tokens. Punctuation marks such as '.' and '!' become separate tokens.
w.lower() ... w.isalnum()	Lower-cases every token (case folding) and keeps only tokens made of letters or digits, which removes punctuation.
stop_words = set(stopwords.words('english'))	Loads the English stop word list into a set. A set gives fast O(1) membership tests.
[w for w in words if w not in stop_words]	A list comprehension that keeps only the words that are not stop words.
PorterStemmer(), SnowballStemmer('english'), LancasterStemmer()	Creates three stemmer objects. stemmer.stem(word) returns the stem of one word, so the program loops over the filtered words and prints all three results side by side for comparison.
File handling part	Writes a sample text.txt, reads and tokenizes it, removes stop words, stems the rest and writes the result to filteredtext.txt. This shows that the same pipeline works on a file.

5. Output

```
Tokens          : ['Programmers', 'program', 'with', 'programming', 'languages', '.', 'The',
    'quick', 'brown', 'foxes', 'are', 'jumping', 'over', 'the', 'lazy', 'dogs', '!']
Cleaned words   : ['programmers', 'program', 'with', 'programming', 'languages', 'the', 'quick',
    'brown', 'foxes', 'are', 'jumping', 'over', 'the', 'lazy', 'dogs']
After stop words: ['programmers', 'program', 'programming', 'languages', 'quick', 'brown',
    'foxes', 'jumping', 'lazy', 'dogs']

Word          Porter      Snowball    Lancaster
programmers   programm    programm    program
program       program     program     program
programming   program     program     program
languages     languag     languag     langu
quick         quick       quick       quick
brown         brown       brown       brown
foxes         fox         fox         fox
jumping       jump        jump        jump
lazy          lazi        lazi        lazy
dogs          dog         dog         dog

File output     : sampl file show stop word filtrat
```

6. Conclusion

The text was tokenized, converted to lower case, cleaned of punctuation and stop words, and stemmed using Porter, Snowball and Lancaster stemmers. Pre-processing reduces the dataset size and keeps only useful, relevant terms, so that the later retrieval or model-building step is more efficient.

7. Viva Questions with Answers

Q1. What is text pre-processing and why is it needed?

Ans: It is the cleaning and normalising of raw text before indexing or modelling. It removes noise (punctuation, case differences, very common words, word variants), reduces the size of the index and improves the accuracy and speed of retrieval.

Q2. What is tokenization?

Ans: Tokenization is breaking a stream of text into smaller units called tokens (usually words or sentences). It is the first step of every NLP and IR pipeline.

Q3. What are stop words? Give examples.

Ans: Stop words are extremely common words that carry little meaning, such as the, is, a, an, in, of, and, to. Search engines ignore them while indexing and searching.

Q4. Why do we remove stop words? Is it always a good idea?

Ans: They waste storage and processing time and do not help to rank documents. It is not always good: phrase queries like 'to be or not to be' or names like 'The Who' lose meaning, so some modern engines keep them.

Q5. What is stemming? Give an example.

Ans: Stemming reduces a word to its root form by removing suffixes using rules. 'connected', 'connecting' and 'connection' all reduce to 'connect'.

Q6. Name some stemming algorithms.

Ans: Porter, Snowball (Porter2), Lancaster (Paice/Husk), Lovins, Dawson, Krovetz and N-gram stemmers.

Q7. What is the difference between Porter, Snowball and Lancaster stemmers?

Ans: Porter is the classic, moderate English stemmer. Snowball is its improved, multi-language version. Lancaster is the most aggressive and fastest, and gives the shortest stems, so it over-stems more often.

Q8. What are over-stemming and under-stemming?

Ans: Over-stemming maps unrelated words to one stem (university and universe to 'univers') - a false positive. Under-stemming fails to map related words to one stem (data and datum) - a false negative.

Q9. What is the difference between stemming and lemmatization?

Ans: Stemming chops suffixes using fixed rules and may produce non-words (lazy gives 'lazi'). Lemmatization uses a dictionary and part of speech to return a real word (the lemma), so it is more accurate but slower.

Q10. Why did the stemmer give 'lazi' for 'lazy' or 'programm' for 'programmers'?

Ans: Stemmers follow suffix rules and do not check a dictionary, so the stem is only a common form used for matching, not necessarily a valid word.

Q11. Why is a Python set used for the stop word list?

Ans: Membership testing in a set takes constant time on average, so checking every word against hundreds of stop words is fast.

Q12. What is case folding?

Ans: Converting all text to lower case so that 'Search', 'SEARCH' and 'search' map to one term.

Q13. Which NLTK data packages are needed for this practical?

Ans: 'punkt' (and 'punkt_tab' in new versions) for word_tokenize, and 'stopwords' for the stop word lists.

Q14. Where is pre-processing used in real systems?

Ans: Search engines, spam filters, sentiment analysis, chatbots, document classification and clustering, and recommendation systems.

Practical 7: Retrieval of Documents using Inverted Files (Inverted Index)

Aim: Implement a program for retrieval of documents using inverted files.

Software / Libraries: Python 3, Jupyter Notebook

1. Theory

An **inverted index** (inverted file) is the core data structure of a search engine. It stores a mapping from every **term** to the list of **documents** (and optionally the positions) in which the term occurs. It is called 'inverted' because a normal (forward) index maps document -> words, while this index maps word -> documents. It works like the index at the back of a book.

Parts of an inverted index

- **Dictionary (vocabulary)** - the sorted list of all distinct terms in the collection.
- **Postings list** - for each term, the list of document IDs that contain it (plus frequency or positions).

Type	What it stores	Use
Record-level inverted index	Only the list of documents for each word	Simple Boolean search, smaller size
Word-level inverted index	Documents AND the position of each word in them	Phrase and proximity queries, more space

Example

Document 1: 'The quick brown fox jumped over the lazy dog'. Document 2: 'The lazy dog slept in the sun'. After tokenizing, the index is: lazy -> {1, 2}, dog -> {1, 2}, quick -> {1}, sun -> {2}, slept -> {2}.

Steps to build an inverted index

- **Fetch the documents** and assign each a unique document ID.
- **Tokenize and normalise** - lower-case, remove punctuation and stop words.
- **Stemming** (optional) - so that 'cat' and 'cats' share one entry.
- **Record document IDs** - for each term add the document ID (and position / frequency) to its postings list.
- **Sort** the dictionary and the postings lists.

Query processing

A query is pre-processed exactly like the documents. For an **AND** query the postings lists of the terms are intersected (only documents containing all terms). For an **OR** query they are united. For **NOT** the documents of the term are removed. Sorted postings lists can be intersected in O(m + n) time with the merge algorithm.

Advantages and disadvantages

- Advantage: very fast full-text search, because only the postings of the query terms are read instead of scanning all documents.
- Advantage: easy to build and widely used (Lucene, Elasticsearch, Google).
- Disadvantage: large storage overhead and extra work when documents are added, updated or deleted.
- Disadvantage: records are returned in the order of the lists, not by usefulness, unless a ranking function (such as TF-IDF) is added.

Other features: index compression (delta / gamma / variable-byte encoding of postings), support for stemming and synonym expansion, Boolean and proximity queries, and multi-language support.

2. Algorithm

1 Create a collection of documents with document IDs.

2 Pre-process each document: lower-case, tokenize, remove stop words.

3 For every term, store the document ID and the position in a dictionary (term -> {doc: [positions]}).

4 Display the inverted index in sorted order.

5 Pre-process the user query in the same way.

6 Fetch the postings of each query term and combine them with AND (intersection) or OR (union).

7 Display the matching document IDs.

3. Program (Python)

```
import re
from collections import defaultdict

# Step 1: Document collection
docs = {
    1: "The quick brown fox jumped over the lazy dog.",
    2: "The lazy dog slept in the sun.",
    3: "A quick brown dog outpaces a quick fox.",
}

STOP_WORDS = {"the", "a", "an", "in", "over", "is", "of", "and", "to"}

# Step 2: Pre-processing (lowercase, tokenize, remove stop words)
def preprocess(text):
    tokens = re.findall(r"[a-z0-9]+", text.lower())
    return [t for t in tokens if t not in STOP_WORDS]

# Step 3: Build a word-level inverted index  term -> {doc_id: [positions]}
inverted_index = defaultdict(dict)
for doc_id, text in docs.items():
    for pos, term in enumerate(preprocess(text), start=1):
        inverted_index[term].setdefault(doc_id, []).append(pos)

# Step 4: Display the index
print("TERM        -> POSTINGS (doc: positions)")
for term in sorted(inverted_index):
    print("{:<11} -> {}".format(term, dict(inverted_index[term])))

# Step 5: Query processing
def search_and(query):
    """Boolean AND: documents that contain ALL the query terms."""
    sets = [set(inverted_index.get(t, {})) for t in preprocess(query)]
    return sorted(set.intersection(*sets)) if sets else []

def search_or(query):
    """Boolean OR: documents that contain ANY of the query terms."""
    sets = [set(inverted_index.get(t, {})) for t in preprocess(query)]
    return sorted(set.union(*sets)) if sets else []

print("\nAND  'quick fox' ->", search_and("quick fox"))
print("OR   'sun fox'   ->", search_or("sun fox"))
print("AND  'lazy dog'  ->", search_and("lazy dog"))
print("AND  'cat'       ->", search_and("cat"))
```

4. Code Explanation

Code / block	What it does
docs = {1: "...", 2: "...", 3: "..."}	A dictionary acting as our document collection: key = document ID, value = document text.
STOP_WORDS and preprocess(text)	preprocess() lower-cases the text, extracts words with the regular expression [a-z0-9]+ (this removes punctuation) and drops stop words.
inverted_index = defaultdict(dict)	A dictionary of dictionaries: term -> {doc_id: [positions]}. defaultdict creates a new empty entry automatically the first time a term is seen.
for pos, term in enumerate(preprocess(text), start=1)	Walks through the words of each document and records in which document and at which position each term occurs. This makes it a word-level inverted index.
print loop over sorted(inverted_index)	Prints the dictionary in alphabetical order with its postings.
search_and(query)	Pre-processes the query, takes the set of document IDs of every term and returns their intersection (documents containing all terms).
search_or(query)	Same, but returns the union of the sets (documents containing any term).
Last four print() calls	Test the search functions, including a term that is not in the index ('cat'), which correctly returns an empty list.

5. Output

```
TERM        -> POSTINGS (doc: positions)
brown       -> {1: [2], 3: [2]}
dog         -> {1: [6], 2: [2], 3: [3]}
fox         -> {1: [3], 3: [6]}
jumped      -> {1: [4]}
lazy        -> {1: [5], 2: [1]}
outpaces    -> {3: [4]}
quick       -> {1: [1], 3: [1, 5]}
slept       -> {2: [3]}
sun         -> {2: [4]}

AND  'quick fox' -> [1, 3]
OR   'sun fox'   -> [1, 2, 3]
AND  'lazy dog'  -> [1, 2]
AND  'cat'       -> []
```

6. Conclusion

An inverted index was built for a small document collection and used to retrieve documents with Boolean AND and OR queries. This shows how search engines find the documents for a term quickly without scanning every document.

7. Viva Questions with Answers

Q1. What is an inverted index?

Ans: A data structure that maps each term to the list of documents in which it occurs. It is the main index used by search engines for fast full-text search.

Q2. Why is it called 'inverted'?

Ans: A forward index maps document -> terms. The inverted index reverses this and maps term -> documents.

Q3. What is the difference between a forward index and an inverted index?

Ans: The forward index lists the words contained in each document and is good for building the index. The inverted index lists the documents containing each word and is good for searching.

Q4. What is a postings list?

Ans: The list of document IDs (often with term frequency or positions) associated with one term in the dictionary.

Q5. Explain record-level and word-level inverted index.

Ans: Record-level stores only which documents contain the word. Word-level also stores the position of the word in each document, which supports phrase and proximity queries but needs more space.

Q6. How is an AND query processed?

Ans: The postings lists of all the query terms are fetched and intersected. With sorted lists a merge in O(m + n) time is used; processing the shortest list first is faster.

Q7. How would you handle a phrase query such as 'lazy dog'?

Ans: Use a positional (word-level) index: find documents containing both words, then check that the position of 'dog' is the position of 'lazy' plus one.

Q8. Why do we remove stop words and apply stemming before indexing?

Ans: Stop word removal makes the index smaller. Stemming merges word variants, so a search for 'cat' also finds 'cats'.

Q9. What are the advantages of an inverted index?

Ans: Fast term lookup, easy to build, supports Boolean and ranked retrieval, can be compressed, and is scalable to billions of documents.

Q10. What are the disadvantages?

Ans: Large storage overhead and high maintenance cost when documents are inserted, updated or deleted.

Q11. How can the size of an inverted index be reduced?

Ans: By compression of postings: store gaps between document IDs (delta encoding) and use gamma or variable-byte codes, plus stop word removal and stemming.

Q12. What data structure in Python behaves like an inverted index?

Ans: A dictionary (hash map) whose keys are terms and values are lists or sets of document IDs.

Q13. What is the difference between Boolean retrieval and ranked retrieval?

Ans: Boolean retrieval returns documents that exactly match the Boolean query (match / no match). Ranked retrieval scores each document, for example with TF-IDF or BM25, and returns them in order of relevance.

Q14. Name some systems that use inverted indexes.

Ans: Apache Lucene, Solr, Elasticsearch, Google and Bing search, and database full-text search features.

Practical 8: Bayesian Network for Medical Data (Heart Disease Diagnosis)

Aim: Write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using the standard Heart Disease Data Set (Python ML library classes / API can be used).

Software / Libraries: Python 3, Jupyter Notebook, pandas, numpy, pgmpy (pip install pgmpy), dataset heart.csv (Cleveland Heart Disease data)

1. Theory

A **Bayesian network** (belief network) is a probabilistic graphical model. It has two parts: (1) a **directed acyclic graph (DAG)** in which each node is a random variable and each edge is a direct conditional dependency, and (2) a set of **conditional probability distributions (CPDs / CPTs)**, one for every node, giving the probability of the node for every combination of its parent values.

Key formulas

- **Bayes theorem:** P(Cause | Evidence) = P(Evidence | Cause) x P(Cause) / P(Evidence).
- **Joint probability (chain rule of the network):** P(X1, X2, ..., Xn) = product over all i of P(Xi | Parents(Xi)). This is why the network needs far fewer numbers than a full joint table.
- **Goal of inference:** compute the posterior P(unobserved cause | observed evidence), for example P(HeartDisease | age, cholesterol).

Example from the manual

A computer does not start (evidence). Two possible causes: electricity failure and computer malfunction. The DAG has edges Electricity failure -> Computer failure and Computer malfunction -> Computer failure. We then compute which cause is more probable given that the computer failed.

Learning a Bayesian network

- **Structure learning** - deciding the nodes and edges (from domain knowledge or algorithms such as hill climbing). In this practical the structure is given by us.
- **Parameter learning** - estimating the CPDs from data. **Maximum Likelihood Estimation (MLE)** uses the relative frequency of each outcome in the data. Bayesian estimation adds a prior (smoothing).

Inference

Variable Elimination is an exact inference method: it sums out the variables that are neither query nor evidence one by one, reusing intermediate factors. Approximate methods such as sampling are used for very large networks.

The data set (Cleveland Heart Disease)

303 patients, 14 commonly used attributes: age, sex, cp (chest pain type 1-4), trestbps (resting blood pressure), chol (serum cholesterol), fbs (fasting blood sugar > 120 mg/dl), restecg (resting ECG 0-2), thalach (maximum heart rate), exang (exercise induced angina), oldpeak (ST depression), slope, ca (number of major vessels 0-3), thal (3 normal, 6 fixed defect, 7 reversible defect) and **heartdisease**, the class label: 0 = no presence, 1 to 4 = presence of disease.

Note: This model is for learning only and must not be used for real medical diagnosis.

2. Algorithm

1. Read heart.csv with pandas and replace '?' by NaN, then drop missing rows.
2. Rename the label column to 'heartdisease' and discretise continuous columns (age, trestbps, chol, thalach) into a few groups.
3. Define the network structure as a list of directed edges.

4 Learn the CPDs from the data with Maximum Likelihood Estimation (model.fit).

5 Create a VariableElimination inference object from the model.

6 Query P(heartdisease | evidence) for different evidence values and print the probability tables.

3. Program (Python)

```
import numpy as np
import pandas as pd
from pgmpy.inference import VariableElimination
try:
    from pgmpy.models import DiscreteBayesianNetwork as BayesianModel  # new pgmpy
except ImportError:
    from pgmpy.models import BayesianModel                             # old pgmpy

# Step 1: Read the Cleveland Heart Disease data set
heart = pd.read_csv('heart.csv')
heart = heart.replace('?', np.nan).dropna()
# some copies of the data call the last column 'target' or 'num'
heart = heart.rename(columns={'target': 'heartdisease', 'num': 'heartdisease'})

print('Few examples from the dataset are given below')
print(heart.head())

# Step 2: Discretise continuous columns (CPDs need a small set of states)
heart['age'] = pd.cut(heart['age'], bins=[0, 40, 55, 70, 120],
                      labels=[0, 1, 2, 3]).astype(int)
heart['trestbps'] = pd.cut(heart['trestbps'], bins=[0, 120, 140, 400],
                           labels=[0, 1, 2]).astype(int)
heart['chol'] = pd.cut(heart['chol'], bins=[0, 200, 240, 700],
                       labels=[0, 1, 2]).astype(int)
heart['thalach'] = pd.cut(heart['thalach'], bins=[0, 120, 150, 250],
                          labels=[0, 1, 2]).astype(int)

# Step 3: Define the structure (nodes and directed edges) of the network
model = BayesianModel([
    ('age', 'trestbps'), ('age', 'fbs'),
    ('sex', 'trestbps'), ('exang', 'trestbps'),
    ('trestbps', 'heartdisease'), ('fbs', 'heartdisease'),
    ('heartdisease', 'restecg'), ('heartdisease', 'thalach'),
    ('heartdisease', 'chol'),
])

# Step 4: Learn the CPDs from data. fit() uses Maximum Likelihood Estimation
# by default (older pgmpy: fit(heart, estimator=MaximumLikelihoodEstimator)).
print('\nLearning CPD using Maximum Likelihood Estimation')
model.fit(heart)

# Step 5: Inference with Variable Elimination
infer = VariableElimination(model)

print('\n1. Probability of HeartDisease given Age group = 2 (age 55-70)')
print(infer.query(variables=['heartdisease'], evidence={'age': 2}))

print('\n2. Probability of HeartDisease given cholesterol group = 2 (>240 mg/dl)')
print(infer.query(variables=['heartdisease'], evidence={'chol': 2}))

print('\n3. Probability of HeartDisease given exang = 1 and fbs = 1')
print(infer.query(variables=['heartdisease'], evidence={'exang': 1, 'fbs': 1}))
```

4. Code Explanation

Code / block	What it does
pd.read_csv('heart.csv') / replace('?', np.nan).dropna()	Loads the data and removes rows with missing values ('?' is used for missing values in the Cleveland file).
rename(columns={'target': 'heartdisease', ...})	Different copies of the data call the last column 'target' or 'num'. The program renames it so the model can use the name 'heartdisease'.
pd.cut(...)	Divides continuous values into bins, e.g. age into 0-40, 40-55, 55-70, 70+. Discrete Bayesian networks need a small number of states per variable; with raw ages the CPD tables would be huge and mostly empty.
BayesianModel([...edges...])	Defines the DAG. Edges such as ('age', 'trestbps') mean 'age influences blood pressure'; ('heartdisease', 'chol') means the disease influences the cholesterol reading. The try/except import supports both old (BayesianModel) and new (DiscreteBayesianNetwork) pgmpy versions.
model.fit(heart)	Learns all the CPDs from the data. By default pgmpy uses Maximum Likelihood Estimation (counts divided by totals).
VariableElimination(model)	Creates the exact inference engine for the learned network.
infer.query(variables=['heartdisease'], evidence={'age': 2})	Asks: what is the probability distribution of heartdisease when the patient's age group is 2? The result is a table with one probability for each of the 5 states (0 to 4).
Differences from the lab manual	The manual uses the continuous values (age = 28) directly and the old class names (BayesianModel, MaximumLikelihoodEstimator). This version discretises the data and uses names that work in current pgmpy, so it runs without errors.

5. Output

Output format (your numbers will depend on heart.csv). Each query prints a table like this:

```
1. Probability of HeartDisease given Age group = 2 (age 55-70)
+-----------------+---------------------+
| heartdisease    |   phi(heartdisease) |
+=================+=====================+
| heartdisease(0) |              0.xxxx |
| heartdisease(1) |              0.xxxx |
| heartdisease(2) |              0.xxxx |
| heartdisease(3) |              0.xxxx |
| heartdisease(4) |              0.xxxx |
+-----------------+---------------------+
(the five probabilities add up to 1)
```

6. Conclusion

A Bayesian network was constructed for the Cleveland heart disease data, its conditional probability tables were learned using maximum likelihood estimation, and variable elimination was used to compute the probability of heart disease for given evidence such as age or cholesterol level.

7. Viva Questions with Answers

Q1. What is a Bayesian network?

Ans: A probabilistic graphical model that represents random variables as nodes of a directed acyclic graph and their conditional dependencies as edges, with a conditional probability table for each node.

Q2. Why must the graph be acyclic?

Ans: A cycle would make a variable depend on itself, so the joint probability could not be factorised into conditional probabilities. A DAG guarantees a valid ordering of variables.

Q3. What is a CPD / CPT?

Ans: Conditional Probability Distribution (Table): the probability of a node taking each value for every combination of values of its parent nodes. Nodes without parents have a simple prior probability.

Q4. State Bayes theorem.

Ans: P(A | B) = P(B | A) x P(A) / P(B). It lets us update the probability of a cause after seeing evidence.

Q5. How does a Bayesian network represent the joint probability?

Ans: P(X1,...,Xn) = product of P(Xi | Parents(Xi)). Each variable depends only on its parents, which reduces the number of probabilities to store.

Q6. What is conditional independence?

Ans: A and B are conditionally independent given C if P(A | B, C) = P(A | C). Once C is known, B gives no extra information about A. The structure of the DAG encodes these independences (d-separation).

Q7. What is Maximum Likelihood Estimation?

Ans: A parameter learning method that chooses the CPD values that make the observed data most probable. For discrete data it equals the relative frequency, e.g. P(fbs = 1 | age = 2) = (patients with age 2 and fbs 1) / (patients with age 2).

Q8. What is Variable Elimination?

Ans: An exact inference algorithm that sums out (eliminates) the hidden variables one at a time, multiplying and combining factors, to obtain the posterior of the query variable.

Q9. What are evidence and query variables?

Ans: Evidence variables are observed values (e.g. age = 2). Query variables are the unknown quantities we want the probability of (heartdisease).

Q10. Why did we discretise age, cholesterol and blood pressure?

Ans: pgmpy's discrete networks need a finite number of states. Continuous values would create enormous CPTs with very few samples per row, giving unreliable estimates.

Q11. Difference between a Bayesian network and Naive Bayes?

Ans: Naive Bayes is a special Bayesian network where the class is the only parent of every feature (features assumed independent given the class). A general Bayesian network allows any dependency structure between variables.

Q12. What are the advantages of Bayesian networks?

Ans: They handle uncertainty and missing evidence, are easy to interpret, combine expert knowledge with data and can answer many different queries.

Q13. What are the limitations?

Ans: CPT size grows exponentially with the number of parents, structure learning is hard, and results depend on the quality of the data and chosen structure.

Q14. What do the values 0 to 4 of the heartdisease attribute mean?

Ans: 0 means no heart disease; 1, 2, 3 and 4 indicate increasing degrees or types of disease presence (angiographic disease status).

Q15. What is the difference between structure learning and parameter learning?

Ans: Structure learning finds the graph (which edges exist). Parameter learning, given the graph, estimates the probability tables.

Q16. Name some real applications of Bayesian networks.

Ans: Medical diagnosis, spam filtering, fault diagnosis, risk analysis, genetics and decision support systems.

Practical 9: Agglomerative Hierarchical Clustering

Aim: Implement the Agglomerative hierarchical clustering algorithm using an appropriate data set (Credit Card data set).

Software / Libraries: Python 3, Jupyter Notebook, pandas, numpy, matplotlib, scikit-learn, scipy, dataset CC_GENERAL.csv (Kaggle credit card data)

1. Theory

Clustering is an unsupervised learning technique that groups data points so that points in the same cluster are more similar to each other than to points in other clusters. It needs no class labels.

Type of clustering	Idea	Example
Connectivity-based	Builds clusters from distances between points	Hierarchical clustering
Centroid-based	Clusters formed around centre points	K-Means, K-Modes
Distribution-based	Data assumed to come from probability distributions	Gaussian Mixture Model
Density-based	Clusters are dense regions separated by sparse regions	DBSCAN

Hierarchical clustering

Hierarchical clustering builds a tree of clusters. There are two kinds. **Agglomerative** (bottom-up): start with every point as its own cluster and repeatedly merge the two closest clusters until one cluster remains. **Divisive** (top-down): start with one cluster of all points and keep splitting. Agglomerative is the more common. It does not need the number of clusters in advance.

Linkage (how the distance between two clusters is measured)

- **Single linkage** - smallest distance between any two points of the clusters (can form long chains).
- **Complete linkage** - largest distance between points of the two clusters.
- **Average linkage** - average distance between all pairs of points.
- **Ward linkage** - merges the pair of clusters that gives the smallest increase in total within-cluster variance; used in this practical.

Dendrogram

A **dendrogram** is the tree diagram of the merges. Leaves are individual points; the height of a joining line is the distance at which clusters were merged. Drawing a horizontal line across the longest vertical gap that is not crossed by a merge gives the best number of clusters (the number of vertical lines it cuts).

Silhouette score

The silhouette of a point is s = (b - a) / max(a, b), where a is the average distance to points in its own cluster and b is the average distance to the points of the nearest other cluster. It ranges from -1 to +1. A higher average silhouette means better-separated clusters. The k with the highest score is chosen.

Pre-processing used

- **StandardScaler** - makes every feature have mean 0 and standard deviation 1 so no feature dominates the distance.
- **normalize** - scales each row to unit length.
- **PCA (Principal Component Analysis)** - reduces the many features to 2 components so that clusters can be plotted and clustered faster.

Data set: CC_GENERAL.csv holds the usage behaviour of about 9000 credit card holders with 18 behavioural features such as BALANCE, PURCHASES, CASH_ADVANCE, CREDIT_LIMIT and PAYMENTS. **Assumption:** every point is initially a cluster of its own and similar points can be merged.

2. Algorithm

1. Import the libraries and load the data; drop the CUST_ID column.
2. Handle missing values (fill with the column median).
3. Scale the features with StandardScaler and normalise them.
4. Reduce the data to 2 dimensions with PCA.
5. Draw the dendrogram using Ward linkage to estimate the number of clusters.
6. Build AgglomerativeClustering models for k = 2 to 6 and plot each clustering.
7. Compute the silhouette score for every k and choose the k with the highest score.

3. Program (Python)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.cluster import AgglomerativeClustering
from sklearn.preprocessing import StandardScaler, normalize
from sklearn.metrics import silhouette_score
import scipy.cluster.hierarchy as shc

# Step 1: Load the Credit Card data set and drop the ID column
X = pd.read_csv('CC_GENERAL.csv')
X = X.drop('CUST_ID', axis=1)

# Step 2: Handle missing values (fill with the column median)
X = X.fillna(X.median())

# Step 3: Scale and normalise so that all features are comparable
X_scaled = StandardScaler().fit_transform(X)
X_normalized = normalize(X_scaled)

# Step 4: Reduce to 2 dimensions with PCA (for plotting and clustering)
pca = PCA(n_components=2)
X_principal = pd.DataFrame(pca.fit_transform(X_normalized), columns=['P1', 'P2'])

# Step 5: Dendrogram (Ward linkage) to choose the number of clusters
plt.figure(figsize=(8, 8))
plt.title('Dendrogram (Ward linkage)')
shc.dendrogram(shc.linkage(X_principal, method='ward'))
plt.show()

# Step 6: Build agglomerative models for k = 2..6 and plot each result
ks = [2, 3, 4, 5, 6]
silhouette_scores = []
for k in ks:
    model = AgglomerativeClustering(n_clusters=k, linkage='ward')
    labels = model.fit_predict(X_principal)
    silhouette_scores.append(silhouette_score(X_principal, labels))

    plt.figure(figsize=(6, 6))
    plt.scatter(X_principal['P1'], X_principal['P2'], c=labels, cmap='rainbow')
    plt.title('Agglomerative clustering, k = {}'.format(k))
    plt.show()

# Step 7: Compare the silhouette scores and pick the best k
for k, s in zip(ks, silhouette_scores):
    print('k = {}  silhouette score = {:.4f}'.format(k, s))
plt.bar(ks, silhouette_scores)
plt.xlabel('Number of clusters')
plt.ylabel('Silhouette score')
plt.show()
print('Best number of clusters:', ks[int(np.argmax(silhouette_scores))])
```

4. Code Explanation

Code / block	What it does
pd.read_csv('CC_GENERAL.csv'); X.drop('CUST_ID', axis=1)	Loads the data and drops the customer ID, because an ID is only a label and has no meaning for clustering.
X.fillna(X.median())	Replaces missing values in every column with that column's median. (The manual uses fillna(method='ffill'), which is deprecated in recent pandas.)
StandardScaler().fit_transform(X); normalize(...)	Scales features to the same range and then normalises each row. Distance-based algorithms such as clustering need this.
PCA(n_components=2)	Projects the data onto the 2 directions with the highest variance; the result is stored in a DataFrame with columns P1 and P2.
shc.dendrogram(shc.linkage(X_principal, method='ward'))	linkage() performs the hierarchical merging and returns the merge history. dendrogram() draws it as a tree.
AgglomerativeClustering(n_clusters=k, linkage='ward')	Creates the model that merges clusters bottom-up until k clusters are left. fit_predict() returns the cluster label of each point.
plt.scatter(..., c=labels, cmap='rainbow')	Plots each point and colours it by its cluster label.
silhouette_score(X_principal, labels)	Measures how well separated the clusters are; collected for every k and shown as a bar chart.
np.argmax(silhouette_scores)	Picks the k with the highest silhouette score as the optimal number of clusters.

5. Output

Output format (values depend on CC_GENERAL.csv). The program shows a dendrogram, five scatter plots and a bar chart, and prints:

```
k = 2  silhouette score = 0.xxxx
k = 3  silhouette score = 0.xxxx
k = 4  silhouette score = 0.xxxx
k = 5  silhouette score = 0.xxxx
k = 6  silhouette score = 0.xxxx
Best number of clusters: 2
(the manual's result for this data set was k = 2)
```

6. Conclusion

Agglomerative hierarchical clustering with Ward linkage was implemented on the credit card data set. The dendrogram and the silhouette scores were used to find the optimal number of clusters (2 for the manual's data).

7. Viva Questions with Answers

Q1. What is clustering?

Ans: Grouping similar objects into clusters without using class labels. It is an unsupervised learning technique.

Q2. What is hierarchical clustering?

Ans: A clustering method that creates a hierarchy (tree) of clusters, shown as a dendrogram, instead of one flat partition.

Q3. Explain agglomerative clustering.

Ans: A bottom-up method. Each point starts as its own cluster. At each step the two closest clusters are merged until all points are in one cluster or the required number of clusters is reached.

Q4. Difference between agglomerative and divisive clustering?

Ans: Agglomerative merges small clusters upwards (bottom-up); divisive starts with one big cluster and splits it downwards (top-down).

Q5. What is a dendrogram and how do we read it?

Ans: A tree diagram of the merge sequence. The height of a join is the distance at which two clusters merged. Cutting the tree with a horizontal line at the largest vertical gap gives the number of clusters.

Q6. Name the linkage methods.

Ans: Single, complete, average and Ward linkage (also centroid).

Q7. What is Ward linkage?

Ans: It merges the two clusters whose merger causes the smallest increase in the total within-cluster variance, which gives compact, similar-sized clusters.

Q8. How is the number of clusters chosen?

Ans: From the dendrogram (longest vertical gap) and by comparing silhouette scores for different k. The k with the highest silhouette score is selected.

Q9. What is the silhouette score?

Ans: s = (b - a) / max(a, b) for each point, where a is the mean distance within its cluster and b the mean distance to the nearest other cluster. Values near +1 mean good clustering, near 0 overlapping clusters, negative means wrong assignment.

Q10. Why do we scale the data before clustering?

Ans: Clustering uses distances. A feature with large values (like credit limit) would dominate a feature with small values, so all features are scaled to the same range.

Q11. Why is PCA used here?

Ans: To reduce the dimensions to two so that the data can be visualised and clustered faster, while keeping most of the variance.

Q12. Difference between K-Means and hierarchical clustering?

Ans: K-Means needs k in advance, is fast on large data and forms spherical clusters. Hierarchical does not need k in advance and gives a dendrogram but is slower (at least O(n squared) time and memory).

Q13. What are the advantages and disadvantages of agglomerative clustering?

Ans: Advantages: no need to specify k beforehand, informative dendrogram, any distance measure. Disadvantages: slow and memory hungry for large data, merges cannot be undone, sensitive to noise and outliers.

Q14. What distance measures can be used?

Ans: Euclidean, Manhattan, cosine and others. Ward linkage requires Euclidean distance.

Q15. What is the role of the CUST_ID column and why was it dropped?

Ans: It is only an identifier of the customer. It carries no behavioural information, so it would distort the distances.

Q16. Where is clustering used?

Ans: Customer segmentation, document clustering, image segmentation, anomaly detection and gene analysis.

Practical 10: PageRank Algorithm

Aim: Implement the PageRank algorithm (using Python / BeautifulSoup / NetworkX).

Software / Libraries: Python 3, Jupyter Notebook, NetworkX (pip install networkx), optional BeautifulSoup for crawling links

1. Theory

PageRank (PR) is a link analysis algorithm used by Google Search to measure the importance of web pages. It is named after Larry Page, a co-founder of Google. Idea: *a page is important if many important pages link to it.* The number and the quality of incoming links decide the rank, not the page content.

The random surfer model

PageRank is the probability that a person randomly clicking on links will arrive at a particular page. With probability d (the damping factor, usually 0.85) the surfer follows a link on the current page, and with probability 1 - d jumps to a random page. The values of all pages form a probability distribution and add up to 1.

Formula

PR(u) = (1 - d) / N + d x SUM over every page v that links to u of PR(v) / L(v)

- PR(u) = PageRank of page u; N = total number of pages.
- d = damping factor (0.85); (1 - d) / N is the share every page receives from random jumps.
- L(v) = number of outbound links of page v. A page divides its own rank equally among its outlinks.

Simplified example (from the manual)

Four pages A, B, C, D, each starting with PR = 0.25. B links to A and C, C links to A, D links to A, B and C. In the first iteration (without damping) A receives 0.25/2 from B + 0.25/1 from C + 0.25/3 from D = 0.125 + 0.25 + 0.083 = 0.458.

Computation by power iteration

PageRank is computed by repeating the formula (iterations) until the values stop changing (difference below a tolerance). It is the principal eigenvector of the Google matrix, and the process is a Markov chain.

Important terms

- **Dangling node** - a page with no outgoing links. Its rank would be lost, so it is treated as linking to all pages equally.
- **Damping factor** - prevents rank from being trapped in loops (spider traps) and guarantees convergence.
- **Personalization vector** - lets the random jump favour certain pages.

Role of BeautifulSoup: to rank real web pages, BeautifulSoup (with requests) is used to download pages and extract the hyperlinks (a tags), which become the edges of the graph. In this practical a small ready-made graph is used.

2. Algorithm

1. Represent the web as a directed graph (page -> list of pages it links to).
2. Initialise the PageRank of every page to 1/N.
3. For every page compute (1 - d)/N + d x sum of PR(v)/L(v) over all pages v linking to it.
4. Repeat until the total change between two iterations is below the tolerance.
5. Print the pages in decreasing order of PageRank.
6. Verify the result using the library function networkx.pagerank().

3. Program (Python)

```
import networkx as nx

# ---------- Part A: PageRank from scratch (power iteration) ----------
def pagerank(links, d=0.85, max_iter=100, tol=1e-6, verbose=False):
    pages = list(links)
    N = len(pages)
    pr = {p: 1.0 / N for p in pages}          # equal initial rank

    for it in range(1, max_iter + 1):
        new_pr = {}
        for p in pages:
            # sum of PR(v)/L(v) for every page v that links to p
            incoming = sum(pr[v] / len(links[v])
                           for v in pages if p in links[v])
            new_pr[p] = (1 - d) / N + d * incoming
        if verbose and it == 1:
            print("After iteration 1:", {k: round(v, 4) for k, v in new_pr.items()})
        diff = sum(abs(new_pr[p] - pr[p]) for p in pages)
        pr = new_pr
        if diff < tol:
            print("Converged after", it, "iterations")
            break
    return pr

# Web graph from the theory example: B->A,C   C->A   D->A,B,C
# Page A is given one outlink (A->D) so that it is not a dangling node.
links = {
    "A": ["D"],
    "B": ["A", "C"],
    "C": ["A"],
    "D": ["A", "B", "C"],
}

print("Without damping (d=1), first iteration only:")
pagerank(links, d=1.0, max_iter=1, verbose=True)

print("\nWith damping factor d = 0.85:")
ranks = pagerank(links, d=0.85)
for page, score in sorted(ranks.items(), key=lambda x: -x[1]):
    print("  PageRank({}) = {:.4f}".format(page, score))

# ---------- Part B: Using the NetworkX library ----------
G = nx.DiGraph()
for src, targets in links.items():
    for t in targets:
        G.add_edge(src, t)

nx_ranks = nx.pagerank(G, alpha=0.85)
print("\nNetworkX PageRank:")
for page, score in sorted(nx_ranks.items(), key=lambda x: -x[1]):
    print("  PageRank({}) = {:.4f}".format(page, score))
```

4. Code Explanation

Code / block	What it does
def pagerank(links, d=0.85, max_iter=100, tol=1e-6)	links is a dictionary: page -> list of pages it links to. d is the damping factor, max_iter the maximum iterations and tol the convergence limit.
pr = {p: 1.0 / N for p in pages}	Gives every page the same starting rank (equal probability).
incoming = sum(pr[v] / len(links[v]) for v in pages if p in links[v])	Adds up the rank contributed by every page v that links to p. Each page v gives PR(v) divided by its number of outlinks L(v).
new_pr[p] = (1 - d) / N + d * incoming	The PageRank formula: random-jump part plus the link-following part.
diff = sum(abs(new_pr[p] - pr[p]) ...); if diff < tol: break	Convergence test. When the ranks no longer change noticeably, the algorithm stops.
pagerank(links, d=1.0, max_iter=1, verbose=True)	Runs only one iteration without damping to reproduce the manual's value A = 0.4583.
The links dictionary	Same graph as the theory (B to A,C; C to A; D to A,B,C). Page A is given one link to D so that it is not a dangling node.
nx.DiGraph() and nx.pagerank(G, alpha=0.85)	Builds a directed graph with NetworkX and calls the library implementation (alpha is the damping factor). The results match our own function, which verifies it.

5. Output

```
Without damping (d=1), first iteration only:
After iteration 1: {'A': 0.4583, 'B': 0.0833, 'C': 0.2083, 'D': 0.25}

With damping factor d = 0.85:
Converged after 24 iterations
  PageRank(A) = 0.3475
  PageRank(D) = 0.3329
  PageRank(C) = 0.1878
  PageRank(B) = 0.1318

NetworkX PageRank:
  PageRank(A) = 0.3475
  PageRank(D) = 0.3329
  PageRank(C) = 0.1878
  PageRank(B) = 0.1318
```

6. Conclusion

The PageRank of every page of a small web graph was calculated using power iteration, and the result was verified with the NetworkX library. Pages that receive links from many (or important) pages obtain a higher rank.

7. Viva Questions with Answers

Q1. What is PageRank?

Ans: An algorithm used by Google to rank web pages by importance, based on the number and quality of links pointing to them.

Q2. Who developed PageRank?

Ans: Larry Page and Sergey Brin, the founders of Google, at Stanford University. It is named after Larry Page.

Q3. What is the basic idea?

Ans: A page is important if it is linked to by other important pages. A link is like a vote, and votes from important pages count more.

Q4. Write the PageRank formula.

Ans: PR(u) = (1 - d)/N + d x sum of PR(v)/L(v) over all pages v that link to u.

Q5. What is the damping factor and why is it 0.85?

Ans: It is the probability that the surfer continues clicking links; with probability 1 - d the surfer jumps to a random page. 0.85 is the value suggested by the original authors. It avoids rank sinks and ensures convergence.

Q6. What is the random surfer model?

Ans: A model of a user who keeps clicking random links and sometimes jumps to a random page. The PageRank of a page is the long-run probability that the surfer is on that page.

Q7. What is a dangling node (dead end)?

Ans: A page with no outgoing links. It would absorb rank, so its rank is distributed equally to all pages.

Q8. What is a spider trap?

Ans: A group of pages that link only to each other, so that the surfer gets trapped and all rank accumulates there. Damping solves this.

Q9. How is PageRank computed?

Ans: By iterating the formula (power iteration) from an initial equal rank until it converges. Mathematically it is the principal eigenvector of the Google matrix.

Q10. What is the sum of the PageRank values of all pages?

Ans: 1 when the probability-distribution version is used (or N in the original version).

Q11. Why does page A get the highest rank in the example?

Ans: Because all other pages (B, C, D) link to it, so it receives the largest total share of rank.

Q12. Is PageRank the only factor in Google's ranking?

Ans: No. Google uses hundreds of signals (content relevance, freshness, location, etc.). PageRank was the first and is the best-known.

Q13. Difference between PageRank and HITS?

Ans: PageRank gives each page one score. HITS gives two scores: a hub score (links to good authorities) and an authority score (linked by good hubs).

Q14. What is the use of BeautifulSoup in this practical?

Ans: To crawl pages and extract hyperlinks so that the link graph for real web pages can be built.

Q15. What is a personalization vector?

Ans: A probability vector that biases the random jump towards chosen pages, used for personalised or topic-sensitive PageRank.

Q16. Name other applications of PageRank.

Ans: Ranking social network users, citation analysis of papers, recommender systems and finding important nodes in any network.

General Information Retrieval Questions

Frequently asked in oral / practical examinations

Q1. What is Information Retrieval?

Ans: IR is the science of finding material (usually documents) of an unstructured nature that satisfies an information need from within large collections, such as web pages and text files.

Q2. Difference between IR and a database system?

Ans: A database stores structured data and answers exact queries with exact results. IR deals with unstructured text and returns ranked, approximately relevant results.

Q3. What are the main IR models?

Ans: Boolean model, vector space model and probabilistic model (plus language models and modern neural models).

Q4. What is TF-IDF?

Ans: Term frequency x inverse document frequency. TF is how often a term occurs in a document; IDF = log(N / df) lowers the weight of terms that occur in many documents. A high TF-IDF means the term is important for that document.

Q5. What are precision and recall?

Ans: Precision = relevant retrieved / total retrieved. Recall = relevant retrieved / total relevant in the collection. F-measure is their harmonic mean.

Q6. What is the vector space model?

Ans: Documents and queries are represented as vectors of term weights (TF-IDF) and ranked by cosine similarity between the query vector and each document vector.

Q7. What is a web crawler?

Ans: A program (spider) that automatically follows hyperlinks to download web pages for indexing.

Q8. Name the parts of a search engine.

Ans: Crawler, indexer (inverted index), query processor, ranking module (e.g. PageRank, TF-IDF) and user interface.

Q9. What is web mining? Its types?

Ans: Applying data mining to web data. Types: web content mining, web structure mining (links, e.g. PageRank) and web usage mining (click logs).

Q10. Which practicals use which concept?

Ans: Practical 6 - pre-processing; 7 - indexing and Boolean retrieval; 8 - probabilistic reasoning (Bayesian network); 9 - clustering of documents or data; 10 - link analysis for ranking.

Quick revision: one line per practical

Practical	One-line summary
6	Tokenize, lower-case, remove punctuation and stop words, then stem (Porter / Snowball / Lancaster).
7	Build an inverted index term -> {doc: positions} and answer AND / OR queries by intersecting or uniting postings.
8	Define a DAG, learn CPDs with MLE, and query P(heartdisease | evidence) with Variable Elimination.
9	Scale, apply PCA, draw a dendrogram (Ward), run agglomerative clustering for k = 2..6, choose k by silhouette score.
10	PR(u) = (1 - d)/N + d x sum PR(v)/L(v); iterate until convergence; verify with nx.pagerank().